

GENERAL STUDY HELPER BOT

Documentation and Project by Jomana Shaltout

Contents

Overview	3
Project Concepts and Plan	3
Project Processing	4
Project’s Present and Missing Features	4
Conclusion	5

Overview

The General Study Helper Bot is a simple chatbot that supports students in their studies. It gives automated messages on how to manage time, deal with stress, take notes and study effectively, and how to stop procrastination.

Project Concepts and Plan

In this program, the user will enter a phrase/question which will trigger an automated response from the bot. Below is an initial flowchart:

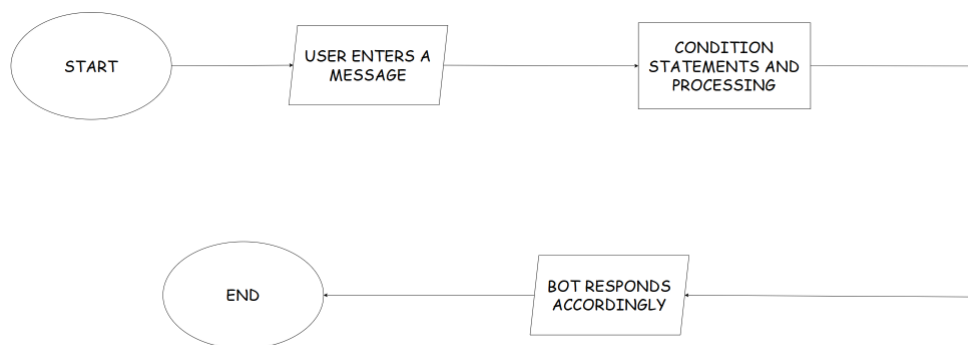


Figure 1: Initial Flowchart

This flowchart was created based on if the program would've been created using conditional statements like match-case (which I initially thought was different from using if-elif-else chain). However, after researching a bit more, the flowchart changed which had changed the approach of the program. Below is the new flowchart:

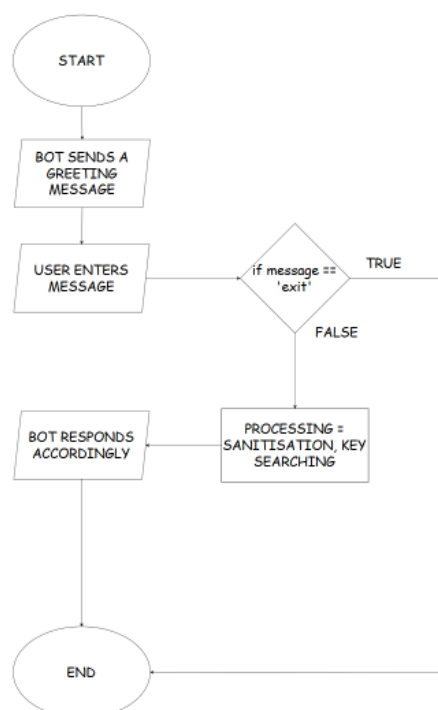


Figure 2: Final Flowchart

This flowchart displays a much better plan as it now shows that a loop will run until the user inputs 'exit'.

Project Processing

Now that there are a clear plan and some concepts to the project, it's time to talk about the processing. First, the data is stored in a Python Dictionary.

What data, and why in a dictionary? The data here is the keyword and the automated responses the bot will choose from depending on the user's input. A dictionary was used as it has a better time complexity than conditional statements.

Second, the program is an IPO (Input-Process-Output) program, this means that there should be some processing done on the input before an output message is generated. Our message input is in string form, so sanitisation will be performed on the string.

What is sanitisation? Sanitisation is the process of cleaning the data so that the bot could easily check it. Because users are unpredictable, an input could come through as 'HELLO ', but the bot may not process it as it is stored as 'hello' instead of 'HELLO'. Sanitisation here helps strip any excess leading or trailing spaces (using the .strip() method), and it also de-capitalises our string (using the .lower() method) to change our 'HELLO ' to 'hello' before performing any other processing.

So, how is the dictionary used for the string values? Every specific keyword holds an automatic response. For example:



```
"hello" : "Hello!",
```

Figure 3: Dictionary Code

The figure above shows that if the user types 'hello', it will respond back with 'Hello!'. Here, 'hello' is a keyword that's searched for in the user input. The program uses the 'in' operator to check whether the keyword is present in the dictionary and if it is, it checks whether it's present in the message input. This allows the bot to respond with the specified message, or otherwise if the keyword is not present in the dictionary.

Project's Present and Missing Features

Below are two lists that contain the present features of the project, and the missing features.

Present Features

- A continuous loop that keeps the chatbot alive.
- A clean termination condition.
- Input sanitisation to handle case differences and accidental leading/trailing spaces.

Missing Features

- Responds only the first keyword found and ignores the second one.
- Only supports exact keyword matching, it will not understand synonyms or misspellings.

- 7 intents covered: hello, studying effectively, exam stress, time management, note taking, procrastination, bye.
- Keyword scanning using in operator.
- Fallback response for unrecognised inputs.
- Greeting message on startup.
- No follow-up handling.
- No handling for empty input
- No input length limit.
- No goodbye message before breaking.
- Study-related topics are limited.

Conclusion

This project helped me learn how a chatbot naturally responds to a user, and how much further a chatbot can improve to ease the user experience.